

Curso Tecnologia em ADS
Sistemas Operacionais

Laboratório: Mecanismos de Sincronização entre Processos

Professor: Eraldo Silveira e Silva — Material elaborado com suporte do ChatGPT

Laboratório: Produtor–Consumidor

Mutex, variáveis de condição e semáforos POSIX

Exercício elaborado com apoio de formatação do ChatGPT

Objetivo

Implementar e analisar três soluções para o problema clássico do produtor–consumidor, utilizando diferentes mecanismos de sincronização.

O laboratório procura distinguir dois problemas fundamentais da programação concorrente:

- **Exclusão mútua:** evitar condições de corrida (*race conditions*) no acesso aos dados compartilhados.
- **Sincronização por disponibilidade de recursos:** impedir que o produtor deposite itens em um buffer cheio ou que o consumidor retire itens de um buffer vazio.

Descrição geral do problema

Considere duas tarefas que compartilham um **buffer circular** de tamanho `N`:

- **Produtor:** gera uma sequência de números inteiros de 1 até `TOTAL` e os deposita no buffer.
- **Consumidor:** retira os itens do buffer, respeitando a ordem em que foram produzidos (FIFO).

O buffer deverá ser implementado utilizando um vetor de inteiros e variáveis que permitam controlar as posições de entrada e saída e a quantidade de itens armazenados.

Utilize inicialmente:

```
1 #define N 5
2 #define TOTAL 20
```

O programa deverá apresentar os valores produzidos e consumidos e, quando aplicável, a quantidade de posições ocupadas no buffer.

As três implementações deverão utilizar as mesmas características gerais, permitindo comparar os mecanismos de sincronização empregados.

1 Produtor–Consumidor utilizando mutex

Implemente o problema produtor–consumidor utilizando exclusivamente um mutex como mecanismo de sincronização. Utilize o tipo `pthread_mutex_t`.

Requisitos

1. Criar uma thread produtora e uma thread consumidora utilizando `pthread_create()`.
2. Implementar um buffer circular de tamanho `N`.
3. Utilizar um único mutex para proteger as operações necessárias sobre os dados compartilhados.
4. Impedir que o produtor deposite um item quando o buffer estiver cheio.
5. Impedir que o consumidor retire um item quando o buffer estiver vazio.
6. Liberar o mutex quando uma operação não puder ser realizada, permitindo que a outra thread prossiga.
7. Proteger individualmente cada operação de inserção ou remoção.
8. Não utilizar semáforos, variáveis de condição, `sched_yield()`, `sleep()` ou `usleep()`.

Questões para análise

1.1. Identifique as variáveis compartilhadas entre as threads. Quais delas podem provocar condições de corrida? Justifique.
1.2. Explique como o mutex evita condições de corrida no acesso ao buffer e às variáveis de controle.
1.3. O mutex também resolve o problema da disponibilidade de recursos? Explique como o programa trata as situações de buffer cheio e buffer vazio.
1.4. O que aconteceria se o produtor mantivesse o mutex adquirido enquanto aguardasse uma posição livre? Analise também o caso equivalente do consumidor.
1.5. A solução utiliza espera ativa (*busy waiting*)? Explique seu funcionamento e suas consequências para a utilização da CPU.
1.6. Para produzir ou consumir um item, quantas vezes uma thread pode precisar adquirir o mutex? Justifique.

2 Produtor–Consumidor utilizando mutex e variáveis de condição

Modifique a implementação do Exercício 1 para utilizar um mutex e duas variáveis de condição (`pthread_cond_t`).

As variáveis de condição deverão representar as seguintes situações:

- `nao_cheio`: existência de espaço disponível para o produtor.
- `nao_vazio`: existência de pelo menos um item disponível para o consumidor.

Requisitos

1. Manter o buffer circular e as duas threads do exercício anterior.
2. Utilizar o mutex para proteger os acessos necessários aos dados compartilhados.
3. Utilizar `pthread_cond_wait()` para bloquear o produtor quando o buffer estiver cheio.
4. Utilizar `pthread_cond_wait()` para bloquear o consumidor quando o buffer estiver vazio.

5. Utilizar `pthread_cond_signal()` para sinalizar alterações que possam permitir o prosseguimento da outra thread.

6. Não utilizar semáforos, espera ativa, `sched_yield()`, `sleep()` ou `usleep()`.

Questões para análise

2.1. Qual é a função do mutex nessa implementação? Qual é a função das variáveis de condição?
2.2. Explique o que acontece com o mutex quando uma thread executa `pthread_cond_wait()`.
2.3. Por que a condição de espera deve ser verificada utilizando `while`, e não simplesmente `if`?
2.4. A execução de `pthread_cond_signal()` garante que a thread sinalizada execute imediatamente? Justifique.
2.5. Compare esta implementação com a do Exercício 1 quanto à espera ativa e ao consumo de CPU.
2.6. O uso de variáveis de condição elimina a necessidade do mutex? Justifique.

3 Produtor–Consumidor utilizando semáforos

Implemente uma terceira versão do problema utilizando semáforos POSIX (`sem_t`).
Nesta implementação, deverão ser utilizados três semáforos:

- `vagas`: representa a quantidade de posições livres no buffer.
- `itens`: representa a quantidade de itens disponíveis para consumo.
- `exclusao`: semáforo binário utilizado para garantir exclusão mútua.

Requisitos

1. Utilizar `sem_init()`, `sem_wait()`, `sem_post()` e `sem_destroy()`.
2. Inicializar corretamente os três semáforos.
3. Bloquear o produtor quando não houver posições disponíveis no buffer.
4. Bloquear o consumidor quando não houver itens disponíveis.
5. Garantir exclusão mútua nas operações necessárias sobre o buffer.
6. Não utilizar mutex, variáveis de condição, espera ativa, `sched_yield()`, `sleep()` ou `usleep()`.

Questões para análise

3.1. Quais devem ser os valores iniciais dos três semáforos? Justifique.
3.2. Explique como os semáforos `vagas` e `itens` controlam a disponibilidade de recursos.
3.3. Qual é a função do semáforo `exclusao`? Ele resolve o mesmo problema que os outros dois semáforos?
3.4. Analise o que poderia acontecer se o produtor executasse `sem_wait(&exclusao)` antes de `sem_wait(&vagas)`. Considere também a situação equivalente no consumidor.
3.5. Considerando apenas um produtor e um consumidor, seria possível eliminar o semáforo `exclusao`, mantendo somente `vagas` e `itens`? Justifique.

3.6. Caso o programa fosse modificado para utilizar múltiplos produtores e múltiplos consumidores, quais cuidados adicionais seriam necessários?

4 Análise comparativa e experimentação

Após concluir os três exercícios, realize os seguintes experimentos.

Experimento 1 — Verificação funcional

Execute cada implementação com `N = 5` e `TOTAL = 20`.

Verifique se todos os itens produzidos são consumidos, se a ordem FIFO é respeitada e se não ocorrem inserções ou remoções inválidas.

Experimento 2 — Tamanho do buffer

Execute as três implementações utilizando:

Parâmetro	Valores
`N`	1, 5 e 20
`TOTAL`	100000

Desabilite as mensagens de produção e consumo durante as medições.

Experimento 3 — Tempo de execução

Utilize o comando:

```
3 time ./pc
```

Registre os tempos `real`, `user` e `sys` para cada implementação e tamanho do buffer. Realize pelo menos cinco execuções por configuração e calcule a média dos tempos obtidos.

Questões finais

4.1. Compare as três implementações quanto à utilização de espera ativa e espera bloqueante.

4.2. Qual é a diferença entre utilizar um mutex para garantir exclusão mútua e utilizar um mecanismo de sincronização para aguardar a disponibilidade de recursos?

4.3. Compare os tempos de execução obtidos. Os resultados confirmam suas expectativas? Explique.

4.4. Qual é a principal diferença conceitual entre variáveis de condição e semáforos contadores?

4.5. Considerando os resultados e as características de cada mecanismo, discuta as vantagens e limitações das três soluções.